

# 2025년 3회 정보처리기사 실기 복원 문제 (해설 추가본)

출처: <https://chobopark.tistory.com/558> (Life-Journey)

## 1번

다음은 UML ( ) 다이어그램이다. 아래 내용을 보고 다이어그램의 명칭을 작성하시오.

시스템을 폴더 모양의 ( ) 단위로 구분하여 구성 요소 간의 관계를 표현하는 UML 구조 다이어그램이다. 하나의 ( ) 안에는 여러 클래스나 하위 ( )가 포함될 수 있으며, «import», «access», «merge» 등의 관계를 통해 의존성을 표현한다.

정답: 패키지

해설:  
패키지(Package) 다이어그램은 시스템을 **폴더 모양의 패키지 단위로 묶어** 구조와 의존 관계를 보여주는 UML 구조 다이어그램입니다.

- '패키지'란 관련 있는 클래스나 요소들을 하나로 묶은 그룹(폴더)입니다. 규모가 큰 시스템을 보기 쉽게 정리하는 역할을 합니다.
- 패키지 안에 또 다른 하위 패키지를 넣을 수 있습니다.
- «import», «access», «merge» 같은 표시(스테레오타입)로 패키지 간 의존성을 나타냅니다.

UML 다이어그램은 크게 **구조(정적)**와 **행위(동적)**로 나뉩니다. 패키지 다이어그램은 **구조 다이어그램**에 속하며, 그림에서 폴더 모양이 보이면 패키지 다이어그램입니다.

## 2번

다음은 소프트웨어 테스트 기법 중 하나에 대한 설명이다. 보기에서 고르시오.

결정 포인트 내에 존재하는 모든 개별 조건식을 대상으로 하는 커버리지 기준. 각각의 조건식이 True와 False 두 경우를 모두 한 번 이상 만족하도록 테스트 케이스를 설계해야 한다. 다만 전체 결정식의 결과가 모두 수행됨을 보장하지는 않는다.

[보기] ㄱ. 경로 ㄴ. 결정 ㄷ. 조건/결정 ㄹ. 변경 조건/결정(MC/DC) ㄴ. 다중 조건 ㄷ. 문장 ㄷ. 분기 ㄹ. 조건 ㄹ. 루프

정답: ㄹ (조건 커버리지)

해설:  
조건(Condition) 커버리지는 결정문(if 등) 안의 **각 개별 조건식이 참(True)과 거짓(False)을 모두** 갖도록 테스트하는 기준입니다.

- 예: if ( A && B ) 라는 결정문이 있으면, A를 True/False 둘 다, B를 True/False 둘 다 만들면 됩니다.
- 단, 개별 조건만 챙기다 보면 **전체 결정식의 결과(참/거짓)는 모두 검증되지 않을 수 있습니다.** 이것이 조건 커버리지의 한계입니다.

비교 포인트:  
- **결정(분기) 커버리지:** 전체 결정식의 참/거짓에 주목.  
- **조건 커버리지:** 개별 조건 하나하나의 참/거짓에 주목.

"개별 조건식의 True/False"라는 키워드가 나오면 조건 커버리지입니다.

## 3번

다음은 유닉스/리눅스 환경에서 자주 사용하는 기본 명령어에 대한 설명이다. 각 설명에 맞는 명령어를 보기에서 골라 연결하시오.

1. 현재 작업 중인 디렉터리의 경로를 출력
2. 디렉터리의 내용을 목록으로 표시
3. 다른 디렉터리로 이동
4. 파일을 복사

[보기] ls, cd, cp, pwd

정답: 1. pwd / 2. ls / 3. cd / 4. cp

해설:

리눅스/유닉스의 가장 기본적인 명령어들입니다. 각 명령어의 **영어 약자 의미**를 알면 외우기 쉽습니다.

- **pwd** (Print Working Directory): 현재 내가 있는 디렉터리(폴더)의 전체 경로를 출력. "나 지금 어디 있지?"
- **ls** (List): 현재 디렉터리 안의 파일·폴더 목록을 보여줌. "여기 뭐 있지?"
- **cd** (Change Directory): 다른 디렉터리로 이동. "다른 폴더로 가자"
- **cp** (Copy): 파일을 복사. "파일 복사하자"

추가로 알아두면 좋은 명령어: mv(이동/이름변경), rm(삭제), mkdir(폴더 생성), chmod(권한 변경). 약자의 뜻과 연결하면 자연스럽게 외워집니다.

## 4번

다음은 오류검출 방식을 설명하는 내용이다. 빈칸 ㉠~㉤에 들어갈 알맞은 용어를 보기에서 고르시오.

㉠ 코드는 여러 개의 검사 비트를 추가하여 오류를 검출하고 수정까지 가능하며, 수신 측에서 자체 수정하는 ㉡ 방식에 속한다. 오류 발생 시 재전송을 요구하는 방식은 ㉢이라 하며, 대표적 검출 기법으로 ㉣ 검사와 ㉤ 검사가 있다.

[보기] ㉠ CRC ㉡ FEC ㉢ BEC ㉣ NAK ㉤ Parity ㉥ MD5 ㉦ BCD ㉧ Hamming

정답: ㉠ ㉧ Hamming / ㉡ ㉡ FEC / ㉢ ㉢ BEC / ㉣ ㉤ Parity / ㉤ ㉠ CRC

해설:  
오류 제어 방식은 수신 측에서 직접 고치느냐(전진), 다시 보내달라고 하느냐(후진)로 나뉩니다.

- **Hamming(해밍) 코드**: 검사 비트를 여러 개 추가해 오류 검출 + 정정(수정)까지 가능. 1비트 오류를 스스로 고칠 수 있습니다.
- **FEC(Forward Error Correction, 전진 오류 수정)**: 수신 측이 재전송 없이 스스로 오류를 정정하는 방식. 해밍 코드가 여기 속합니다.
- **BEC(Backward Error Correction, 후진 오류 수정)**: 오류가 나면 송신 측에 재전송을 요청하는 방식.
- **Parity(패리티) 검사**: 1의 개수가 짝수/홀수가 되도록 비트 1개를 추가해 오류를 검출하는 간단한 방법.
- **CRC(순환 중복 검사)**: 다항식 나눗셈의 나머지를 이용한 강력한 오류 검출 방법.

핵심 정리: 스스로 고침=FEC(해밍), 재전송=BEC. 검출 기법=패리티·CRC. "정정까지 가능 = 해밍"이 단골 키워드입니다.

## 5번

다음은 C코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

struct Test {
    int i;
    const char *g;
};

int main() {
    struct Test test[] = {{1, "AB"}, {2, "DC"}, {3, "EB"}};
    struct Test *p = &test[1];
    printf("%s", p->g + (p->i - 1));
    return 0;
}
```

정답: C

해설:  
구조체 배열과 문자열 포인터 연산이 결합된 문제입니다.

- `p = &test[1]` → p는 두 번째 구조체 {2, "DC"}를 가리킵니다.
- `p->i = 2, p->g = "DC"`(문자열의 시작 주소).

이제 `p->g + (p->i - 1)` 을 계산합니다.  
- `p->i - 1 = 2 - 1 = 1`  
- `p->g + 1` → 문자열 "DC"의 시작 주소에서 **1칸 뒤로** 이동 → 'C'를 가리킴.  
- `%s` 로 출력하면 그 위치부터 끝까지 → **"C"**

핵심: 문자열 포인터에 정수를 더하면 그만큼 뒤 글자로 이동합니다. "DC"에서 +1은 'C'를 가리키고, %s는 거기부터 끝까지 출력하므로 "C"가 됩니다.

## 6번

다음은 C코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

int main(void) {
    char str[] = "REPUBLICOFKOREA";
    int a = 0;
    while (str[a] != '\0')
        ++a;
    putchar(str[a - 2]);
    return 0;
}
```

정답: E

해설:

이 코드는 먼저 문자열의 끝(널 문자)을 찾은 뒤, 끝에서 두 번째 글자를 출력합니다.

- while문은 str[a] 가 널 문자('\0')가 아닐 때까지 a를 증가시킵니다. 즉 a는 문자열 길이만큼 커져서, 반복이 끝나면 a는 널 문자의 위치(= 문자열 길이)를 가리킵니다.
- "REPUBLICOFKOREA"는 15글자이므로 a = 15 (str[15]가 '\0').
- str[a - 2] = str[13]을 출력합니다.

문자열을 인덱스와 함께 보면:

```
R(0) E(1) P(2) U(3) B(4) L(5) I(6) C(7) O(8) F(9) K(10) O(11) R(12) E(13) A(14)
```

str[13] = 'E'

핵심: while로 끝을 찾으면 a는 문자열 길이가 되고, a-2 는 끝에서 두 번째 글자(A 바로 앞의 E)입니다.

## 7번

다음은 C코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

struct Node {
    struct Node* next;
    unsigned int x;
};

int main() {
    struct Node t1 = { 0, 5u };
    struct Node t2 = { 0, 7u };
    struct Node t3 = { 0, 11u };

    t3.next = &t2;
    t2.next = &t1;

    struct Node* curr = &t3;
    int sum = 0;

    while (curr) {
        sum = sum * 3 + curr->x;
        curr = curr->next;
    }

    sum = (sum ^ 42u) + 100u;
    printf("%u\n", sum);
}
```

정답: 187

해설:

연결 리스트 순회 + 비트 XOR 연산이 결합된 문제입니다.

연결 순서: t3(11) → t2(7) → t1(5) → NULL

while문은 각 노드를 돌며 `sum = sum*3 + 현재값` 을 누적합니다:

- 시작 `sum = 0`
- t3(11): `sum = 0*3 + 11 = 11`
- t2(7): `sum = 11*3 + 7 = 33 + 7 = 40`
- t1(5): `sum = 40*3 + 5 = 120 + 5 = 125`

이제 마지막 연산 `sum = (sum ^ 42) + 100` :

- ^ 는 **XOR(배타적 논리합)**입니다. 두 비트가 다르면 1, 같으면 0.
- 125 = 2진수 1111101, 42 = 0101010
- 125 XOR 42 = 1010111 = 87
- `sum = 87 + 100 = 187`

핵심: 연결 리스트를 돌며 누적 계산하고, XOR 연산은 자리별로 "다르면 1"을 적용해 계산합니다.

8번

아래 코드는 Machine 인터페이스를 정의하고 WashingMachine 클래스에서 사용하고자 한다. 빈칸에 들어갈 올바른 키워드를 작성하시오.

```
interface Machine {
    void run();
}

class WashingMachine (____ 빈칸____) Machine {
    private String name;

    public WashingMachine() {
        this.name = "LG Washer";
    }

    public void run() {
        System.out.println("Washing machine running");
    }
}

public class Main {
    public static void main(String[] args) {
        WashingMachine wm = new WashingMachine();
        wm.run();
    }
}
```

정답: implements

해설:

자바에서 **인터페이스를 구현**할 때는 `implements` 키워드를 사용합니다.

- **인터페이스(interface)**는 "이런 메서드를 반드시 만들어라"라고 정해놓은 **약속(설계도)**입니다. 여기서는 Machine이 `run()` 메서드를 가지라고 요구합니다.
- 클래스가 인터페이스를 구현하려면 `class 클래스명 implements 인터페이스명` 형태로 쓰고, 인터페이스가 요구한 메서드(run)를 **반드시 구현**해야 합니다.

비교:

- 클래스 상속: `extends` 키워드 사용 (부모 클래스의 기능을 물려받음).
- 인터페이스 구현: `implements` 키워드 사용 (약속된 메서드를 직접 만들음).

"클래스 상속=extends, 인터페이스 구현=implements"로 구분하세요.

9번

다음은 파이썬에 대한 문제이다. 출력값에 알맞는 값을 작성하시오.

```
data = [
    [3, 5, 2, 4, 1],
    [4, 5, 1],
    [4, 4, 1, 5, 4],
    [4, 5]
]

result = {}

for index, lis in enumerate(data):
    list_sum = sum(lis)
    list_len = len(lis)
    result[index] = (list_sum, list_len)

print(result)
```

[출력값] {0: (①, ②), 1: (③, ④), 2: (⑤, ⑥), 3: (⑦, ⑧)}

정답: ①=15, ②=5, ③=10, ④=3, ⑤=18, ⑥=5, ⑦=9, ⑧=2

해설:  
파이썬의 `enumerate`, `sum`, `len` 을 이용해 각 리스트의 **합과 길이**를 딕셔너리로 정리하는 문제입니다.

- `enumerate(data)` 는 (인덱스, 원소)를 함께 꺼내줍니다. index는 0,1,2,3.
- `sum(lis)` 는 리스트 원소들의 합, `len(lis)` 는 원소 개수.
- `result[index] = (합, 길이)` 튜플로 저장.

각 리스트를 계산합니다:  
- index 0: [3,5,2,4,1] → 합 = 3+5+2+4+1 = **15**, 길이 = **5**  
- index 1: [4,5,1] → 합 = 4+5+1 = **10**, 길이 = **3**  
- index 2: [4,4,1,5,4] → 합 = 4+4+1+5+4 = **18**, 길이 = **5**  
- index 3: [4,5] → 합 = 4+5 = **9**, 길이 = **2**

따라서 {0:(15,5), 1:(10,3), 2:(18,5), 3:(9,2)}.

핵심: `enumerate`는 인덱스와 값을 동시에, `sum`은 합, `len`은 개수를 구합니다. 침착하게 더하면 됩니다.

10번

다음은 테이블에서 조건값을 실행한 화면이다. 알맞는 결과값을 작성하시오. (※ 원문에 이미지 포함)

정답: 4

해설:  
이미지가 포함된 SQL 결과값 문제입니다. 일반적으로 이런 유형은 **WHERE 조건에 맞는 행의 개수를 세거나(COUNT), 특정 집계 결과를 구하는** 문제입니다.  
원문 테이블 이미지를 기준으로, 주어진 조건을 만족하는 행을 하나씩 확인해 세면 결과가 **4**가 됩니다.

- 이런 문제를 풀 때의 일반 전략:
1. WHERE 조건을 정확히 해석한다. (AND는 둘 다 만족, OR는 하나만 만족해도 됨)
  2. 테이블의 각 행이 조건에 맞는지 하나씩 체크한다.
  3. COUNT라면 맞는 행의 수를, 집계함수라면 해당 계산을 수행한다.

조건 연산자(AND/OR)와 IN, 비교연산자(>, >=, =)를 침착하게 적용하는 것이 핵심입니다.

11번

다음 설명에 해당하는 인증 기술을 쓰시오.

한 번 사용하면 즉시 폐기되어 재사용이 불가능하다. 시간 동기화나 카운터 기반 방식으로 매번 새로운 값을 생성하며, 해시 함수를 이용해 서버에 평문을 저장하지 않고 유효성을 확인할 수 있다. 은행 인증 등 고보안 영역에서 널리 사용된다.

정답: OTP

해설:  
OTP(One-Time Password, 일회용 비밀번호)는 한 번만 쓰고 폐기되는 비밀번호입니다.

- 매번 새로운 값이 생성되므로, 누군가 한 번 훔쳐보더라도 다음에는 못 씁니다(재사용 불가).
- 생성 방식은 두 가지입니다:
- **시간 동기화 방식(TOTP):** 일정 시간(예: 30초)마다 값이 바뀜.
- **카운터 기반 방식(HOTP):** 사용 횟수에 따라 값이 바뀜.
- 은행 보안카드, OTP 생성기, 인증 앱 등에서 쓰이는 고보안 인증 기술입니다.

키워드 "한 번 쓰고 폐기 + 매번 새 값 + 시간/카운터 = OTP"입니다.

## 12번

다음은 Java의 상속과 생성자 호출에 관한 코드이다. 밑줄에 알맞은 단어를 작성하시오.

```
class Rectangle {
    int width, height;
    Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }
}

class Square extends Rectangle {
    Square(int a) {
        ____ (a, a);
    }
    int getSquareArea() {
        return width * height;
    }
}

public class Main {
    public static void main(String[] args) {
        Square sq = new Square(10);
        System.out.println(sq.getSquareArea());
    }
}
```

정답: super

해설:  
자바에서 부모 클래스의 생성자를 호출할 때는 `super` 키워드를 사용합니다.

- Square(정사각형)는 Rectangle(직사각형)을 상속받습니다.
- Rectangle에는 `Rectangle(int width, int height)` 생성자만 있고 기본 생성자가 없습니다. 따라서 자식 Square의 생성자에서 반드시 부모 생성자를 명시적으로 호출해야 합니다.
- `super(a, a) →` 부모 Rectangle의 생성자에 width=a, height=a를 넘깁니다. 정사각형이니 가로·세로가 같죠.

참고: `this(...)` 는 같은 클래스의 다른 생성자 호출, `super(...)` 는 부모 클래스 생성자 호출입니다. 여기서는 부모(Rectangle)를 호출해야 하므로 `super`입니다.

이 코드 실행 시 width=height=10이 되어 `getSquareArea()` = 10\*10 = 100이 출력됩니다.

## 13번

다음은 인증 및 자원 접근 방식에 대한 설명이다. 알맞은 단어를 작성하시오.

사용자가 새로운 사이트에 가입하지 않고 평소 이용하던 서비스 계정으로 로그인할 수 있게 해주는 기술. 비밀번호는 전달되지 않으며 승인한 범위에 대해서만 접근 권한이 위임된다. 인증이 아닌 '인가' 절차로 접근 권한을 제3자에게 부여하고, Access Token을 발급한다. 대표적 예는 소셜 로그인이며, SSO와 달리 서로 다른 서비스 간 권한 위임에 초점이 있다.

정답: OAuth

해설:  
OAuth는 비밀번호를 공유하지 않고 다른 서비스에 일부 권한만 위임하는 개방형 표준입니다.

- "구글 계정으로 로그인", "카카오로 로그인" 같은 **소셜 로그인**이 대표적인 OAuth 사례입니다.
- 핵심 개념: **인증(Authentication)**이 아니라 **인가(Authorization)**입니다.
- 인증 = "당신이 누구인지 확인"
- 인가 = "당신에게 무엇을 할 권한을 줄지 결정"
- 사용자가 승인한 **범위(scope)에 한해서만** 제3자 앱에 접근을 허용하고, 그 증표로 **Access Token**을 발급합니다. 비밀번호는 제3자에게 절대 전달되지 않습니다.

비교: SSO(Single Sign-On)는 "한 번 로그인으로 여러 서비스 이용"에 초점, OAuth는 "서로 다른 서비스 간 권한 위임"에 초점. 키워드 "권한 위임 + Access Token + 소셜 로그인 = OAuth"입니다.

## 14번

다음 아래의 테이블을 확인하여 R % S(나누기, division)의 결과를 테이블 형태로 기재하시오. (※ 원문에 테이블 이미지 포함)

정답:  
| A |  
| --- |  
| a1 |

해설:  
관계대수의 **나누기(Division, ÷)** 연산을 묻는 문제입니다. 나누기는 조금 까다로운 개념이라 차근차근 이해해야 합니다.

나누기 연산  $R \div S$ 의 의미: "S에 있는 모든 값과 짝을 이루는 R의 값을 찾아라"입니다.

- 쉬운 예로 설명하면:
- R이 (학생, 과목) 테이블이고 S가 (과목) 테이블이라면,
  - $R \div S$ 는 "S에 있는 모든 과목을 다 수강한 학생"을 골라냅니다.

이 문제에서는 R의 값들 중 S의 모든 원소와 빠짐없이 매칭되는 것이 **a1**뿐이라서, 결과 테이블에 a1만 남습니다.

핵심: 나누기는 "**나누는 쪽(S)의 모든 값을 만족시키는** 항목"을 찾는 연산입니다. '모든'이라는 조건이 핵심입니다.

## 15번

다음은 C코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
int main() {
    int x=7, y=4, z;
    z = y%3<3 ? 2 : 1;
    z = z & z >> 1;
    z = x>5 && z<=3 ? z*x : z/x;
    printf("%d", z);
    return 0;
}
```

정답: 0

해설:  
삼항 연산자와 비트 연산이 섞인 문제입니다. **연산자 우선순위**에 주의하며 한 줄씩 풀니다.

- 1줄:  $z = y\%3 < 3 ? 2 : 1$**   
-  $y \% 3 = 4 \% 3 = 1$ .  $1 < 3$ 은 참  $\rightarrow z = 2$ .

2줄: `z = z & z >> 1`

- 우선순위: 시프트(>>)가 비트AND(&)보다 먼저입니다.
- `z >> 1 = 2 >> 1 = 1` (오른쪽으로 1비트 이동 = 2를 절반으로).
- `z & 1 = 2 & 1 = 10 & 01 = 00 = 0.` → `z = 0.`

3줄: `z = x>5 && z<=3 ? z*x : z/x`

- `x > 5` 는 `7>5` → 참. `z <= 3` 은 `0<=3` → 참. 둘 다 참(&&) → 참.
- 참이므로 `z * x = 0 * 7 = 0.` → `z = 0.`

출력: 0

핵심: ① 삼항 연산자 `조건 ? A : B` (참이면 A, 거짓이면 B), ② 시프트(>>)가 AND(&)보다 우선순위가 높다는 점을 정확히 적용해야 합니다.

## 16번

관계형 데이터베이스 개념에 대한 설명이다. 빈칸에 들어갈 용어를 보기에서 골라 순서대로 쓰시오.

- ㄱ. 테이블에서 한 행(Row)을 의미하며, 하나의 레코드를 구성하는 요소
- ㄴ. 실제 데이터가 저장되어 있는 테이블의 내용 전체를 의미하며, 데이터의 상태를 나타낸다
- ㄷ. 테이블에 저장된 행(Row)의 총 개수를 의미한다

[보기] 스키마, 속성(Attribute), 튜플(Tuple), 차수(Degree), 인스턴스(Instance), 카디널리티(Cardinality)

정답: ㄱ. 튜플 / ㄴ. 인스턴스 / ㄷ. 카디널리티

해설:

관계형 데이터베이스의 기본 용어를 구분하는 문제입니다.

- **튜플(Tuple):** 테이블의 **한 행(Row)**. 하나의 레코드, 즉 한 건의 데이터입니다.
- **인스턴스(Instance):** 어느 시점에 테이블에 **실제로 저장된 데이터 전체**(상태). 릴레이션 인스턴스라고도 합니다.
- **카디널리티(Cardinality):** 테이블의 **행(튜플)의 총 개수**.

함께 정리:

- 속성(Attribute) = 열
- 차수(Degree) = 열의 개수
- 카디널리티 = 행의 개수
- 튜플 = 한 행

"행 = 튜플, 행의 개수 = 카디널리티, 데이터 전체 상태 = 인스턴스"로 구분하세요.

## 17번

다음은 Java에 대한 코드이다. 알맞는 출력값을 작성하시오.

```
enum Tri {
    A("A"), B("AB"), C("ABC");

    private String code;

    Tri(String code) {
        this.code = code;
    }

    public String code() {
        return code;
    }
}

public class Main {
    public static void main(String[] args) {
        Tri t = Tri.values()[Tri.A.name().length()];
        System.out.print(t.code());
    }
}
```



```
}

```

정답: AB

해설:

자바의 열거형(enum)을 다루는 문제입니다. enum은 정해진 상수들의 집합으로, 각 상수에 추가 값(여기선 code)을 붙일 수 있습니다.

핵심 메서드 두 가지:

- values() : enum의 모든 상수를 **배열**로 반환합니다. 여기선 [A, B, C] (인덱스 0,1,2).
- name() : enum 상수의 **이름(문자열)**을 반환합니다. Tri.A.name() = "A".

계산을 따라갑시다:

- Tri.A.name() = "A" → .length() = 1 (글자 수 1개).
- Tri.values()[1] = 배열의 인덱스 1번 = **B**.
- B.code() → B에 붙은 code = **"AB"**.

출력: **"AB"**

핵심: values()는 상수 배열, name()은 상수 이름 문자열. "A".length()가 1이라 values()[1]=B가 되고, B의 code가 "AB"입니다.

## 18번

다음은 정보보안의 접근통제(Access Control) 방식에 대한 설명이다. 보기에서 골라 빈칸에 작성하시오.

- ㄱ. 중앙에서 보안 정책을 일괄 설정하며 주체가 임의로 수정할 수 없다. 군사 기밀, 국가 보안 등 높은 보안 환경에서 사용되며 보안 등급(Top Secret 등)에 따라 접근 여부가 결정된다.
- ㄴ. 직무나 역할(Role)에 따라 접근 권한을 부여하는 방식. 역할에 권한을 묶어 부여하여 관리가 용이하다.
- ㄷ. 자원의 소유자가 접근 권한을 자유롭게 부여·회수할 수 있는 방식. 사용자 임의 설정이 가능해 보안성이 상대적으로 낮다.

[보기] DAC, MAC, RBAC

정답: ㄱ. MAC / ㄴ. RBAC / ㄷ. DAC

해설:

접근통제(Access Control)는 "누가 무엇에 접근할 수 있는가"를 결정하는 방식으로, 3가지가 자주 출제됩니다.

- **MAC(Mandatory Access Control, 강제 접근통제): 중앙(관리자)이 강제로** 정책을 정하며, 사용자가 임의로 바꿀 수 없습니다. 보안 등급(Top Secret 등)에 따라 접근이 결정됩니다. 군사·국가 기밀 같은 고보안 환경용. ('Mandatory=강제')
- **RBAC(Role-Based Access Control, 역할 기반 접근통제): 직무나 역할(Role)에** 권한을 묶어 부여합니다. "팀장 역할에는 이런 권한" 식이라 관리가 편합니다. ('Role=역할')
- **DAC(Discretionary Access Control, 임의 접근통제): 자원 소유자가 자유롭게** 권한을 주고 회수합니다. 유연하지만 보안성은 상대적으로 낮습니다. ('Discretionary=임의/재량')

암기 팁: MAC(강제, 중앙), RBAC(역할), DAC(소유자 임의). 이름의 뜻과 연결하면 외우기 쉽습니다.

## 19번

다음은 테스트케이스의 구성요소에 대한 설명이다. 괄호 ( ) 안에 들어갈 알맞는 보기를 고르시오. (※ 원문에 이미지 포함)

[보기] ㄱ. 테스트 조건 ㄴ. 테스트 환경 ㄷ. 테스트 유형 ㄹ. 테스트 데이터 ㄴ. 예상 결과 ㄷ. 수행 단계 ㄴ. 성공/실패 기준

정답: (왼쪽순으로) ㄱ / ㄹ / ㄴ

해설:

**테스트 케이스(Test Case)**는 소프트웨어가 제대로 동작하는지 확인하기 위한 "시험 문제지"입니다. 보통 다음 요소로 구성됩니다.

- **테스트 조건:** 어떤 상황/조건에서 테스트하는가.
- **테스트 데이터(입력값):** 테스트할 때 넣을 입력 데이터.
- **예상 결과(기대값):** 그 입력에 대해 "이런 결과가 나와야 한다"는 정답.

테스트는 결국 "특정 조건에서 → 어떤 데이터(입력)를 넣으면 → 어떤 결과가 나와야 한다"의 흐름입니다. 그래서 정답이 조건(ㄱ) → 데이터(ㄹ) → 예상 결과(ㄴ) 순서가 됩니다.

핵심: 입력(데이터)을 넣고 기대 결과(예상 결과)와 실제 결과를 비교하는 것이 테스트의 본질입니다.

## 20번

다음은 SQL에 관한 문제이다. 아래 A테이블을 참고하여 쿼리의 결과를 작성하시오. (※ 원문에 테이블 이미지 포함)

```
SELECT count(col2) FROM A WHERE col1 IN (2, 3) OR col2 IN (3, 5)
```

정답: 4

해설:  
COUNT 와 IN , OR 연산자가 결합된 SQL 문제입니다. 조건을 정확히 해석하는 것이 핵심입니다.

조건 분석:  
- col1 IN (2, 3) → col1 값이 2 또는 3인 행.  
- col2 IN (3, 5) → col2 값이 3 또는 5인 행.  
- 두 조건이 OR 로 연결 → 둘 중 하나라도 만족하면 선택됩니다. (AND였다면 둘 다 만족해야 함)

풀이 방법:  
1. 테이블의 각 행에 대해 "col1이 2나 3인가?" 또는 "col2가 3이나 5인가?"를 확인.  
2. 둘 중 하나라도 참이면 그 행을 카운트.  
3. count(col2) 는 선택된 행 중 col2가 NULL이 아닌 행의 개수를 셉니다.

원문 테이블 기준으로 조건을 만족하는 행이 4개여서 결과는 4입니다.

주의 포인트:  
- IN (값1, 값2) 는 "값1 또는 값2 중 하나와 같으면"을 뜻합니다.  
- OR 는 둘 중 하나만 참이어도 선택, AND 는 둘 다 참이어야 선택.  
- COUNT(컬럼) 은 그 컬럼이 NULL인 행은 세지 않습니다. ( COUNT(\*) 는 NULL 포함 전체 행을 셈)